

PFLICHTENHEFT

Projekt: BugTracker

Webanwendung zur Fehlerverwaltung

Projektbezeichnung	BugTracker – Webanwendung zur Fehlerverwaltung
Kurs	Software Engineering II
Hochschule	MCI Management Center Innsbruck
Auftraggeber	Prof. Andrea Corradini
Auftragnehmer	Gruppe DE 3
Team	Amort Julian-Alexander, Demianov Oleksandr, Gulenko Maksym, Lamesic Patrick, Widner Josef
Erstellt am	April 2026
Version	1.2
Status	In Bearbeitung

Inhalt

Änderungshistorie	3
1. Einleitung	4
1.1 Zweck des Dokuments	4
1.2 Geltungsbereich	4
1.3 Beziehung zum Lastenheft	4
1.4 Abkürzungsverzeichnis	4
2. Zielbestimmung	5
2.1 Muss-Kriterien (Must-have)	5
2.2 Soll-Kriterien (Should-have)	5
2.3 Kann-Kriterien (Could-have)	5
2.4 Nicht enthalten (Won't-have)	5
3. Produkteinsatz	6
3.1 Anwendungsbereiche	6
3.2 Zielgruppen/ Personas	6
3.3 Betriebsbedingungen	6
4. Produktübersicht	7
4.1 Package-Struktur Backend	9
4.2 Frontend-Struktur	9
5. Produktfunktionen	10
5.0 Use-Case-Übersicht	10
5.1 FA-01 – Bug anlegen (US-01)	12
5.2 FA-02 – Bugliste anzeigen (US-02)	13
5.3 FA-03 – Bug-Detailansicht (US-03)	14
5.4 FA-04 – Bug bearbeiten (US-04)	14
5.5 FA-05 – Bug archivieren (US-05)	15
5.6 FA-06 – Status verwalten (US-06)	15
5.7 FA-07 – Priorität setzen	17
5.8 FA-08 – Bearbeiter zuweisen (US-08)	17
5.9 FA-09 – Nach Status filtern (US-09)	17
5.10 FA-10 – Nach Titel suchen (US-10, Kann)	18
5.11 FA-11 – Kommentarfunktion (US-11, Kann)	18
5.12 FA-12 – Benutzeranmeldung (US-12)	18
5.12 FA-12 – Benutzeranmeldung (US-12)	19
5.13 FA-13 – E-Mail-Benachrichtigung	20
5.14 FA-14 – Bug-Historie (US-14)	20
5.15 FA-15 – Rollen-basierte Zugriffskontrolle (US-15)	20
5.16 FA-16 – Tags verwalten (US-16)	21
6. Produktdaten	22
6.1 User-Tabelle	22
6.2 Bug-Tabelle	22
6.3 Kommentar-Tabelle	23
6.4 Historien-Tabelle	23

6.5 Tag-Tabelle	24
6.6 Bug-Tag-Junction-Tabelle	24
6.7 Klassendiagramm	25
7. Produktleistungen	26
8. Qualitätsanforderungen	27
9. Benutzungsoberfläche	28
9.1 Allgemeine Gestaltungsrichtlinien	28
9.2 Hauptmasken	28
9.3 Rollenbasierte Sichtbarkeit	28
10. Nichtfunktionale Anforderungen	29
11. Technische Produktumgebung	31
11.1 Software	31
11.2 Hardware	31
11.3 Orgware	31
11.4 Produktschnittstellen	31
12. Spezielle Anforderungen an die Entwicklungsumgebung	32
13. Gliederung in Teilprodukte (Release-Planung)	33
14. Abnahmekriterien	34
14.1 Funktionale Abnahme	34
14.2 Qualitative Abnahme	34
14.3 Dokumentation	34
14.4 Präsentation & Bericht	34
15. Ergänzungen	35
15.1 Annahmen und Abhängigkeiten	35
15.2 Offene Punkte	35
15.3 Lieferumfang	35

Änderungshistorie

Nr.	Datum	Version	Art der Änderung	Verantwortlich
1	28.04.2026	0.1	Erstellung des Grundgerüsts	Lamesic Patrick
2	29.04.2026	0.5	Ausarbeitung Systemarchitektur, Datenmodell, Produktfunktionen	Gulenko Maksym
3	30.04.2026	0.8	Qualitätsanforderungen, UI-Beschreibung, Nicht-funktionale Anforderungen	Demianov Oleksandr
4	01.05.2026	1.0	Finalisierung: Abgrenzung, technische Umgebung, Ergänzungen	Amort J.-A., Widner J.
5	04.05.2026	1.1	Use-Case-Diagramm und Architekturdiagramm ergänzt; Sequenzdiagramm zu FA-01 hinzugefügt; Klassendiagramm und Activity-Klasse korrigiert (action/field/oldValue/newValue); FA-07 Priorität auf Soll korrigiert; FA-13-Abschnitt ergänzt.	Gulenko Maksym
6	10.05.2026	1.2	M:N-Tag-Relation, NFA-07 CSRF-Wortlaut, PR-Branch develop, Package-Prefix at.mci.bugtracker, SessionStore-Architektur dokumentiert	Gulenko Maksym

1. Einleitung

1.1 Zweck des Dokuments

Das vorliegende Pflichtenheft konkretisiert und detailliert das Lastenheft (Version 1.1) für die Webanwendung BugTracker, erstellt durch das Entwicklungsteam (Auftragnehmer) auf Basis der Anforderungen von Prof. Andrea Corradini (Auftraggeber). Es beschreibt das WIE und WOMIT die Umsetzung, ohne Implementierungsentscheidungen auf Code-Ebene vorwegzunehmen. Das Dokument bildet die vertragliche Grundlage für den Auftragsumfang und dient als Referenz während der gesamten Entwicklungsphase.

1.2 Geltungsbereich

Dieses Pflichtenheft gilt für das Projekt BugTracker im Rahmen des Kurses Software Engineering II am MCI Management Center Innsbruck. Es umfasst alle Aspekte der zu entwickelnden Webanwendung, einschließlich Backend (Spring Boot), Frontend (React) und Datenbankschicht (PostgreSQL).

1.3 Beziehung zum Lastenheft

Dieses Pflichtenheft basiert auf dem Lastenheft BugTracker (Version 1.1, Stand 27.04.2026). Alle Anforderungen aus dem Lastenheft werden aufgegriffen, konkretisiert und mit Querverweisen versehen. Die Anforderungs-IDs aus dem Lastenheft (FA-xx, NFA-xx, UC-xx) bleiben erhalten.

1.4 Abkürzungsverzeichnis

Abkürzung	Bedeutung
FA	Funktionale Anforderung
NFA	Nicht-funktionale Anforderung
UC	Use Case (Anwendungsfall)
API	Application Programming Interface
REST	Representational State Transfer
SPA	Single Page Application
CRUD	Create, Read, Update, Delete
MoSCoW	Must / Should / Could / Won't (Priorisierungsmethode)
JWT	JSON Web Token
BCrypt	Passwort-Hashing-Algorithmus
ORM	Object-Relational Mapping
CI/CD	Continuous Integration / Continuous Delivery
LF	Lastenheft-Anforderung (Querverweis)
JDBC	Java Database Connectivity
DAO	Data Access Object
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)

2. Zielbestimmung

2.1 Muss-Kriterien (Must-have)

Die folgenden Anforderungen sind für eine erfolgreiche Abnahme zwingend erforderlich:

- FA-01 (/LF FA-01/): Nutzende können Bugs mit Titel, Beschreibung, Priorität und Status anlegen.
- FA-02 (/LF FA-02/): Die Bugliste zeigt alle erfassten Bugs in einer Übersicht.
- FA-03 (/LF FA-03/): Einzelne Bugs können in einer Detailansicht eingesehen werden.
- FA-04 (/LF FA-04/): Bestehende Bugs können bearbeitet werden.
- FA-05 (/LF FA-05/): Bugs können archiviert (weich gelöscht) werden, um den Audit-Trail zu erhalten.
- FA-06 (/LF FA-06/): Jedem Bug ist ein Status zugeordnet; Statusübergänge folgen einer definierten State Machine.
- FA-12 (/LF FA-12/): Nutzende können sich mit Benutzernamen und Passwort anmelden und abmelden.
- FA-15 (/LF FA-15/): Rollenbasierte Zugriffskontrolle mit den Rollen TESTER, DEVELOPER und ADMIN.

2.2 Soll-Kriterien (Should-have)

Diese Anforderungen sollen – soweit möglich – erfüllt werden:

- FA-07 (/LF FA-07/): Zuweisung einer Priorität (Niedrig, Mittel, Hoch, Kritisch) zu jedem Bug.
- FA-08 (/LF FA-08/): Zuweisung eines Bearbeiters zu jedem Bug.
- FA-09 (/LF FA-09/): Filterung der Bugliste nach Status.
- FA-14 (/LF FA-14/): Anzeige der Änderungshistorie je Bug.
- FA-16 (/LF FA-16/): ADMIN-Nutzer können Tags anlegen, bearbeiten, löschen und Bugs zuweisen.

2.3 Kann-Kriterien (Could-have)

Diese Anforderungen werden umgesetzt, sofern Zeit und Ressourcen es erlauben:

- FA-10 (/LF FA-10/): Freitextsuche nach Bug-Titel in der Bugliste.
- FA-11 (/LF FA-11/): Kommentarfunktion zu einzelnen Bugs.

2.4 Nicht enthalten (Won't-have)

Folgende Punkte werden in der ersten Version noch nicht enthalten sein:

- FA-13 (/LF FA-13/): E-Mail-Benachrichtigungen – wird nicht implementiert.
- Integration mit externen Systemen (Jira, Git Issues, E-Mail-Server).
- Mobile Apps (iOS/Android) – nur Desktop-Browser werden unterstützt.
- Automatische Fehlererkennung oder Logging-Anbindung.
- Mehrsprachigkeit der Oberfläche (nur Deutsch).
- Kanban- oder Sprint-Boards mit Drag-and-Drop.
- Zeiterfassung pro Bug.
- Multi-Tenancy (eine Installation = ein Team).
- Versionierung der Bug-Beschreibung (Edit-Historie pro Feld).
- Externe REST-API für Drittsysteme.

3. Produkteinsatz

3.1 Anwendungsbereiche

Der BugTracker ist eine interne Webanwendung für kleine bis mittelgroße Softwareentwicklungsteams (5–20 Personen). Der Einsatz erfolgt im kaufmännisch-technischen Umfeld eines Softwareprojekts. Die Anwendung läuft lokal oder auf einem einfachen internen Server und wird ausschließlich über Standard-Webbrowser bedient.

3.2 Zielgruppen/ Personas

Die Anwendung richtet sich an drei Nutzergruppen mit unterschiedlichen Rollen:

Rolle	Beispiel-Persona	Hauptaufgaben
TESTER	Tom (24, QA-Werkstudent)	Bugs anlegen, dokumentieren, durchsuchen
DEVELOPER	Marie (28, Backend-Entwicklerin)	Bugs bearbeiten, Status setzen, Kommentare hinzufügen
ADMIN	Sandra (35, Projektleiterin)	Alle Funktionen + Benutzerverwaltung, Tags, System-Konfiguration

3.3 Betriebsbedingungen

- Betriebsumgebung: Desktop-PC in Büroumgebung (Windows, macOS oder Linux).
- Zugriff: über einen modernen Webbrowser (Chrome 120+, Firefox 120+, Edge 120+, Stand 2026).
- Betriebsdauer: Werktags während normaler Arbeitszeiten; kein 24/7-Betrieb erforderlich.
- Netzwerk: Lokales Netzwerk oder localhost; keine Internetverbindung notwendig.
- Gleichzeitige Benutzer: bis zu 20 parallele Sessions.

4. Produktübersicht

Der BugTracker ist eine zweiteilige Webanwendung:

- Backend: REST-API auf Basis von Spring Boot 3.x (Java 21), gesichert durch Spring Security.
- Frontend: Single Page Application (SPA) auf Basis von React 18+ mit TypeScript.
- Datenbank: PostgreSQL für persistente Datenspeicherung.

Die Kernfunktionen der Anwendung umfassen:

- Benutzeranmeldung und rollenbasierte Zugriffskontrolle (TESTER / DEVELOPER / ADMIN).
- Vollständiges Bug-Lifecycle-Management: Anlegen → Bearbeiten → Review → Erledigen → Archivieren.
- Strukturierte Statusverwaltung gemäß einer definierten State Machine.
- Filterung und Suche in der Bugliste.
- Kommentare und Änderungshistorie je Bug.
- Tag-Verwaltung durch Administratoren.

Die Kommunikation zwischen Frontend und Backend erfolgt ausschließlich über JSON-REST-Aufrufe. Das Frontend kann unabhängig vom Backend deployed werden.

Die nachfolgende Abbildung zeigt die Systemarchitektur im Überblick. Sie folgt einer klassischen 3-Schichten-Architektur, in der Frontend und Backend strikt getrennt sind (vgl. NFA-08).

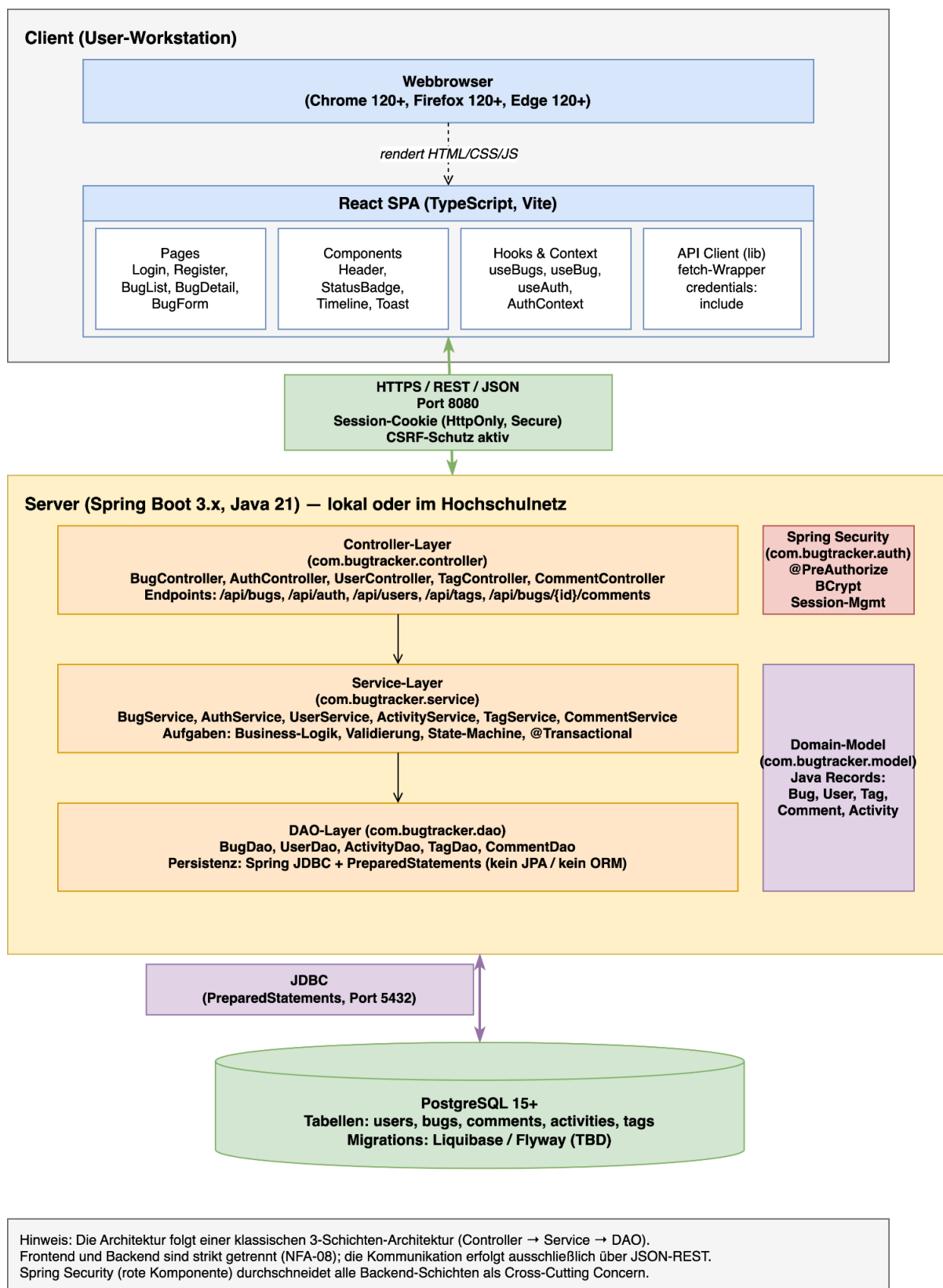


Abbildung 1: Systemarchitektur des BugTracker (3-Schichten-Architektur, Browser → React SPA → Spring Boot → PostgreSQL).

4.1 Package-Struktur Backend

Das Backend folgt einer layered architecture mit folgenden Packages:

- `at.mci.bugtracker.controller` – REST-Controller, HTTP-Mapping
- `at.mci.bugtracker.service` – Business-Logik, Validierungen, Transaktionen
- `at.mci.bugtracker.dao` – Datenbankzugriff via Spring JDBC (NamedParameterJdbcTemplate)
- `at.mci.bugtracker.model` – Java Records (Bug, User, Activity, Comment, Requests-DTOs)
- `at.mci.bugtracker.auth` – Security-Konfiguration, SessionStore, SessionAuthFilter
- `at.mci.bugtracker.db` – JdbcTemplate-Wrapper für DAO-Schicht
- `at.mci.bugtracker.exception` – Custom-Exceptions (EntityNotFoundException, InvalidStatusTransitionException)
- `at.mci.bugtracker.util` – Utilities (HttpException für strukturierte HTTP-Fehler)

4.2 Frontend-Struktur

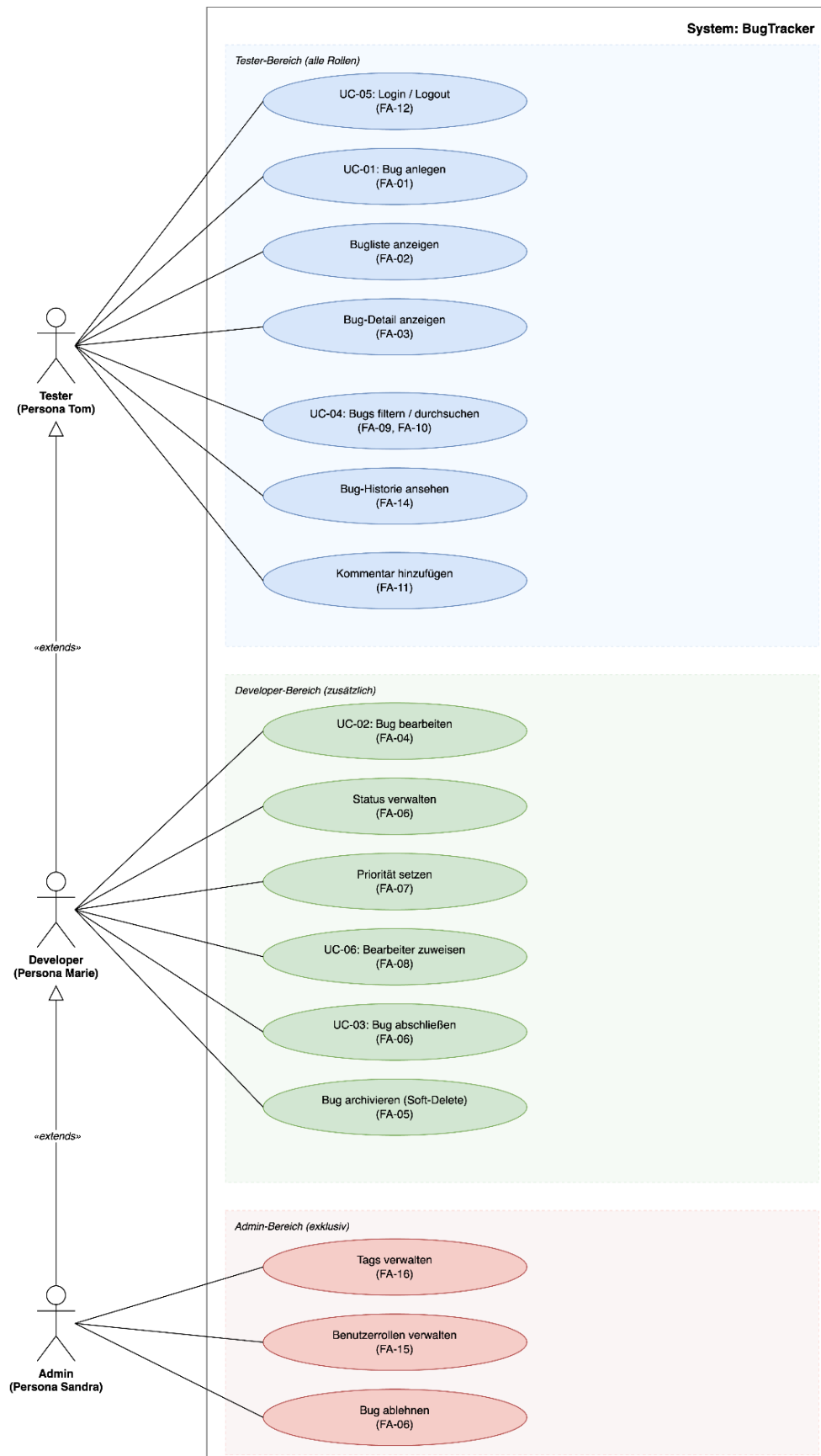
- `src/pages` – Seitenkomponenten (Login, Register, BugList, BugDetail, BugForm)
- `src/components` – Wiederverwendbare UI-Bausteine (Header, StatusBadge, Timeline, Toast)
- `src/lib` – API-Client (fetch-Wrapper mit credentials: include)
- `src/hooks` – Custom Hooks (useBugs, useBug, useAuth)
- `src/context` – AuthContext mit AuthProvider

5. Produktfunktionen

Die folgende Tabelle stellt einen Überblick aller Produktfunktionen dar, geordnet nach FA-ID, User Story und Akzeptanzkriterien.

5.0 Use-Case-Übersicht

Bevor die einzelnen funktionalen Anforderungen im Detail beschrieben werden, gibt das nachfolgende Use-Case-Diagramm einen visuellen Überblick darüber, welcher Akteur welche Funktionen des Systems nutzen kann. Die Rollen-Hierarchie folgt dem UML-Konzept der Generalisierung («extends»): Admin erbt alle Rechte eines Developer, Developer erbt alle Rechte eines Tester. Es sind daher pro Rolle nur die zusätzlich hinzukommenden Use Cases eingezeichnet.

**Legende:**

Blau (Tester-Bereich): für alle Rollen zugänglich — Tester, Developer und Admin (FA-01, FA-02, FA-03, FA-09, FA-10, FA-11, FA-12, FA-14)
 Grün (Developer-Bereich): zusätzlich für Developer und Admin (FA-04, FA-05, FA-06, FA-07, FA-08)
 Rot (Admin-Bereich): ausschließlich für Admin (FA-15, FA-16, Bug ablehnen via FA-08)

Pfeil «extends»: UML-Generalisierung — Admin erbt alle Rechte des Developer, Developer erbt alle Rechte des Tester.
 Es werden daher nur die zusätzlich neuen Use Cases je Rolle eingezeichnet, nicht die geerbten.

Abbildung 2: Use-Case-Diagramm BugTracker mit den drei Akteuren Tester, Developer und Admin sowie ihrer Rollen-Vererbung. Farben markieren den Rollenbereich (Blau = alle, Grün = Developer und Admin, Rot = nur Admin).

5.1 FA-01 – Bug anlegen (US-01)

Attribut	Beschreibung
Verweis Lastenheft	FA-01 (Muss)
User Story	Als Tester (Persona Tom) möchte ich einen neuen Bug erfassen, damit Fehler dokumentiert sind.
Eingabefelder	Titel (Pflicht), Beschreibung (Pflicht), Priorität (optional, Default: MITTEL), Tags (optional, Mehrfachauswahl möglich, Default: keine), Status (automatisch: NEU)
AC1	Erfolgreiches Anlegen: Bug erscheint binnen 2 Sek in der Liste mit Status NEU
AC2	Pflichtfeld Titel fehlt: Fehlermeldung wird angezeigt, Bug wird NICHT gespeichert
AC3	Pflichtfeld Beschreibung fehlt: Fehlermeldung, Bug wird NICHT gespeichert
AC4	Tags auswählen: Ein oder mehrere passende Tags zum jeweiligen Bug auswählbar (Multi-Select); duplikate Zuweisungen werden serverseitig verhindert
AC5	Reporter automatisch: Eingeloggter User wird automatisch als Reporter eingetragen
Backend-Endpunkt	POST /api/bugs
HTTP-Antwort	201 Created + Bug-JSON 400 Bad Request bei Validierungsfehler 401 Unauthorized

Der genaue Ablauf beim Anlegen eines Bugs durch einen eingeloggten Nutzer ist im folgenden Sequenzdiagramm dargestellt. Es zeigt das Zusammenspiel der Frontend- und Backend-Komponenten von der Eingabe im Formular über die REST-Schicht, die Service- und DAO-Schicht im Backend bis zur Persistenz in der Datenbank inklusive automatischem Eintrag in die Bug-Historie.

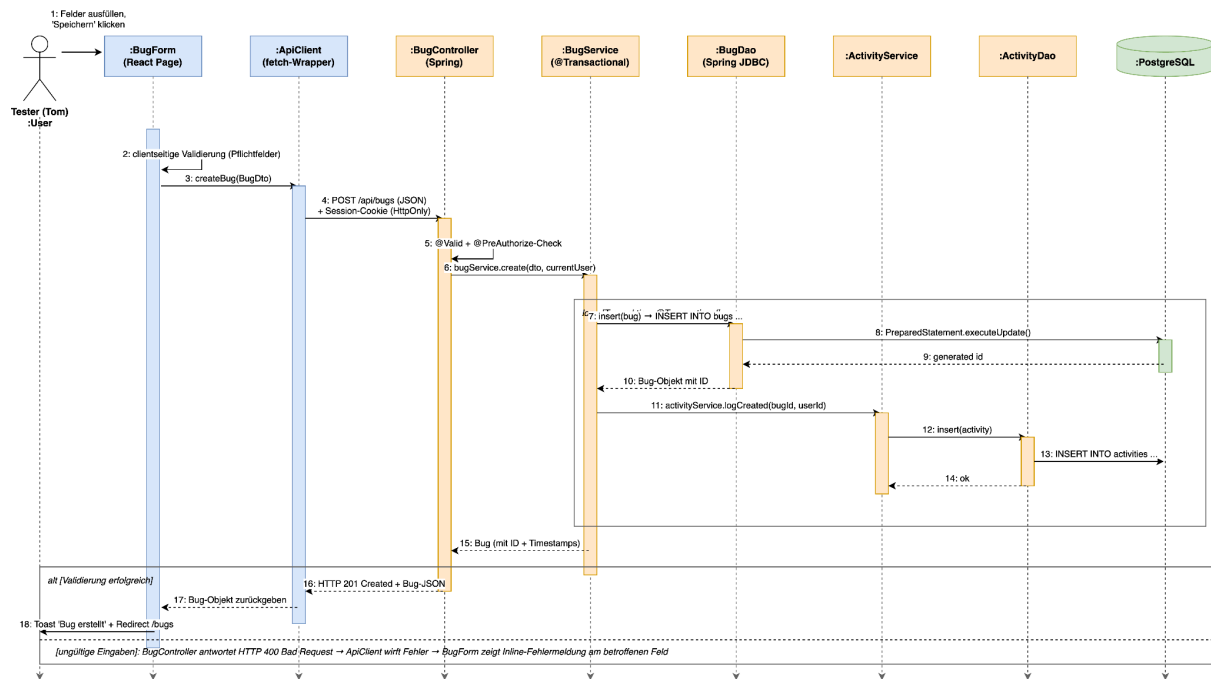


Abbildung 3: Sequenzdiagramm zu UC-01 / FA-01 — Ablauf beim Anlegen eines neuen Bugs.

5.2 FA-02 – Bugliste anzeigen (US-02)

Attribut	Beschreibung
Verweis Lastenheft	FA-02 (Muss)
User Story	Als Projektleiterin (Persona Sandra) möchte ich alle Bugs in einer Übersicht sehen, um den Stand einzuschätzen.
AC1	Standardliste: Alle nicht-archivierten Bugs werden angezeigt
AC2	Sichtbare Spalten: ID, Titel, Status, Priorität, Bearbeiter, Erstelldatum, Tag
AC3	Pagination: Bei mehr als 50 Bugs wird seitenweise navigiert (50 pro Seite)
AC4	Standardsortierung: Neueste Bugs oben (absteigend nach created_at)
AC5	Leerer Zustand: Hinweis "Noch keine Bugs erfasst" + Button "Neuer Bug"
Backend-Endpunkt	GET /api/bugs?status=NEU&page=0&size=50
HTTP-Antwort	200 OK + Bug-Array (JSON) 401 Unauthorized

5.3 FA-03 – Bug-Detailansicht (US-03)

Attribut	Beschreibung
Verweis Lastenheft	FA-03 (Muss)
User Story	Als Entwicklerin (Persona Marie) möchte ich alle Details eines Bugs einsehen vor der Bearbeitung.
AC1	Klick auf Listeneintrag öffnet Detailansicht
AC2	Vollständige Felder: Titel, Beschreibung, Status, Priorität, Tag, Reporter, Bearbeiter, Erstelldatum, letztes Update
AC3	Bug-Historie als Timeline-Komponente sichtbar (wer/wann/was)
AC4	Aktions-Buttons: Bearbeiten, Status ändern, Bearbeiter zuweisen, Archivieren
Backend-Endpunkt	GET /api/bugs/{id}
HTTP-Antwort	200 OK + Bug-Detail-JSON 404 Not Found 401 Unauthorized

5.4 FA-04 – Bug bearbeiten (US-04)

Attribut	Beschreibung
Verweis Lastenheft	FA-04 (Muss)
User Story	Als Entwicklerin (Persona Marie) möchte ich einen Bug aktualisieren mit neuen Erkenntnissen.
AC1	Bearbeiten-Button öffnet vorbefülltes Formular mit aktuellen Werten
AC2	Erfolgreiches Speichern: Detailansicht zeigt sofort neue Werte
AC3	Validierung: Leerer Titel wird mit Fehlermeldung abgelehnt (HTTP 400)
AC4	Historien-Eintrag: Jede Änderung wird automatisch in der Bug-Historie protokolliert
AC5	Abbruch: Klick auf "Abbrechen" verwirft alle Änderungen ohne Speichern
Backend-Endpunkt	PUT /api/bugs/{id}
HTTP-Antwort	200 OK + aktualisierter Bug 400 Bad Request 404 Not Found

5.5 FA-05 – Bug archivieren (US-05)

Attribut	Beschreibung
Verweis Lastenheft	FA-05 (Muss) – Soft-Delete
User Story	Als Projektleiterin (Persona Sandra) möchte ich erledigte Bugs archivieren statt physisch zu löschen.
AC1	Archivieren-Button in der Detailansicht sichtbar
AC2	Bestätigungsdialog: "Bug wirklich archivieren?" erscheint vor Ausführung
AC3	Archivierung setzt Bug.archived = true; Bug verschwindet aus Standardliste
AC4	Im Filter "Archiv" werden archivierte Bugs angezeigt
AC5	Reaktivierung: Reaktivieren-Button setzt archived = false zurück
AC6	Hartes Löschen ausgeschlossen: Bug existiert weiterhin in der Datenbank (Audit-Trail)
Backend-Endpoint	DELETE /api/bugs/{id} (Soft-Delete, setzt archived=true)

5.6 FA-06 – Status verwalten (US-06)

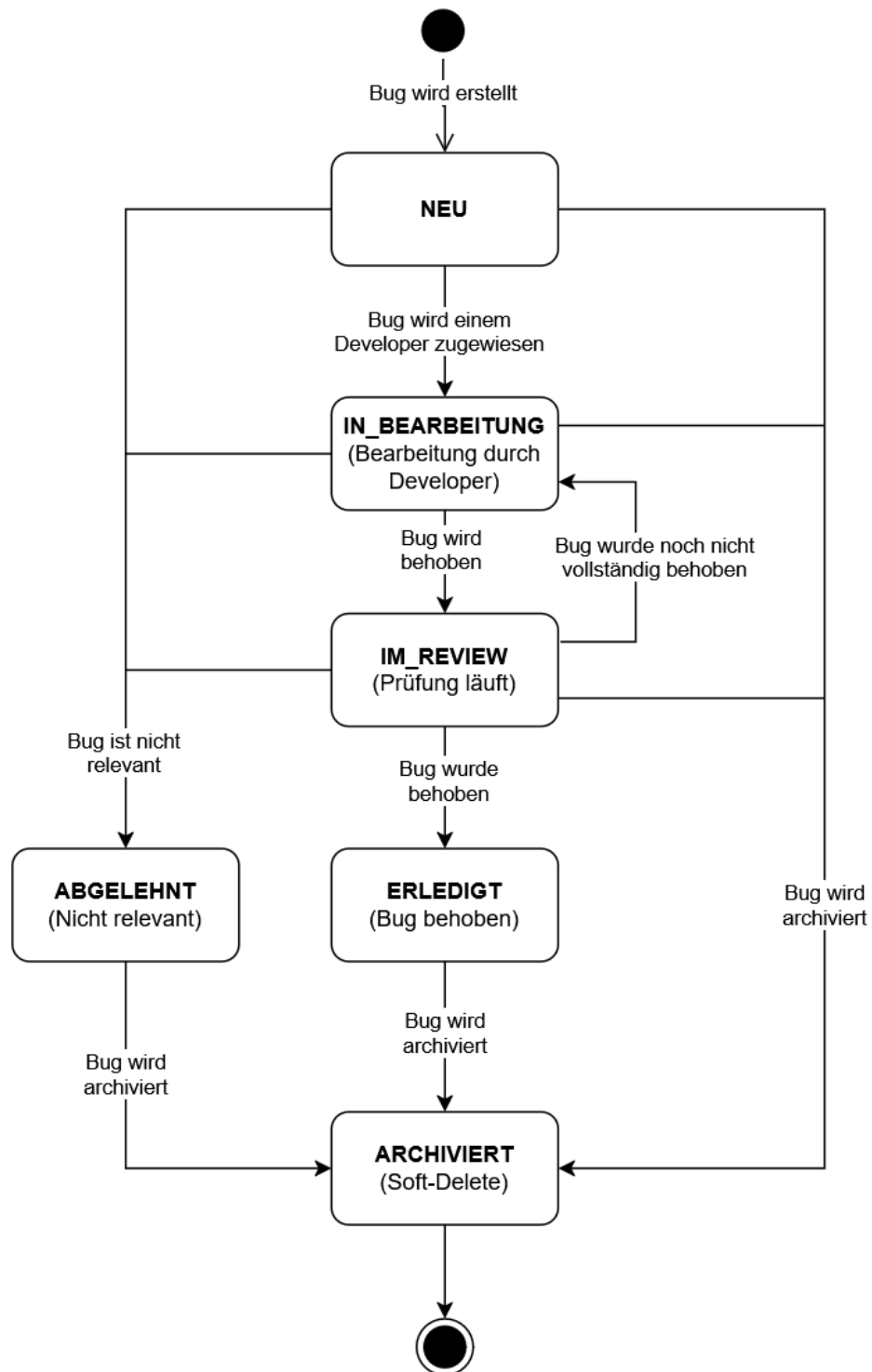
Die Statusverwaltung folgt einer State Machine. Folgende Übergänge sind erlaubt:

Von-Status	Erlaubte Ziel-Status	Hinweis
NEU	IN_BEARBEITUNG, ABGELEHNT	Startpunkt nach Bug-Erstellung
IN_BEARBEITUNG	IM_REVIEW, ABGELEHNT	Entwickler nimmt Bug auf
IM_REVIEW	ERLEDIGT, IN_BEARBEITUNG, ABGELEHNT	Review kann zurückgewiesen werden
ERLEDIGT	ARCHIVIERT	Kein Rückweg nach NEU
ABGELEHNT	ARCHIVIERT	Endgültig abgelehnt
Alle Status	ARCHIVIERT	Archivierung jederzeit möglich

Verbotene Übergänge (z.B. ERLEDIGT → NEU) werden mit HTTP 400 Bad Request abgelehnt und lösen eine Fehlermeldung im Frontend aus. Jeder erlaubte Statuswechsel erzeugt automatisch einen Historien-Eintrag.

Attribut	Beschreibung
Backend-Endpoint	PATCH /api/bugs/{id}/status
HTTP-Antwort	200 OK 400 Bad Request (ungültiger Übergang) 401 Unauthorized

Status verwalten: Zustandsdiagramm



5.7 FA-07 – Priorität setzen

Attribut	Beschreibung
Verweis Lastenheft	FA-07 (Soll)
User Story	Als Entwickler (Persona Marie) und Projektleiterin (Persona Sandra) möchte ich die Priorität eines Bugs ändern können, um die Relevanz neu einzustufen.
AC1	Eindeutiger gültiger Zustand: NIEDRIG, MITTEL, HOCH, KRITISCH
AC2	Die Änderung wird nach dem Speichern in der DB hinterlegt

5.8 FA-08 – Bearbeiter zuweisen (US-08)

Attribut	Beschreibung
Verweis Lastenheft	FA-08 (Soll)
User Story	Als Projektleiterin (Persona Sandra) möchte ich einem Bug einen Bearbeiter zuordnen, damit Verantwortlichkeit klar ist.
AC1	Bearbeiter-Dropdown zeigt alle aktiven User
AC2	Bearbeiter-Name erscheint in Detail- und Listenansicht
AC3	Zuweisung wird automatisch in der Bug-Historie protokolliert
AC4	"Kein Bearbeiter" setzt assigneeld = null; Anzeige zeigt "—"
Backend-Endpunkt	PATCH /api/bugs/{id}/assignee

5.9 FA-09 – Nach Status filtern (US-09)

Attribut	Beschreibung
Verweis Lastenheft	FA-09 (Soll)
User Story	Als Tester (Persona Tom) möchte ich die Bugliste nach Status filtern, um nur relevante Bugs zu sehen.
AC1	Filter-Dropdown mit allen Statuswerten im Listen-Header sichtbar
AC2	Mehrfachauswahl: Mehrere Status gleichzeitig auswählbar
AC3	Sofortige Wirkung: Liste aktualisiert sich innerhalb 1 Sekunde
AC4	URL-teilbar: Filter ist als Query-Parameter in der URL gespeichert
AC5	Reset-Button stellt ungefilterte Liste wieder her
Backend-Endpunkt	GET /api/bugs?status=NEU&status=IN_BEARBEITUNG

5.10 FA-10 – Nach Titel suchen (US-10, Kann)

Attribut	Beschreibung
Verweis Lastenheft	FA-10 (Kann)
User Story	Als Entwicklerin (Persona Marie) möchte ich schnell zu einem bestimmten Bug per Titelsuche kommen.
AC1	Suchfeld im Listen-Header platziert und stets sichtbar
AC2	Echtzeit-Filterung: Liste filtert sich nach 300ms Tipp-Pause automatisch
AC3	Case-insensitive: "login" findet auch "LOGIN BUTTON"
AC4	Treffer-Hervorhebung: Suchbegriff im Titel optisch markiert
AC5	Kombinierbar mit Status-Filter: beide wirken gleichzeitig
Backend-Endpunkt	GET /api/bugs?search=login

5.11 FA-11 – Kommentarfunktion (US-11, Kann)

Attribut	Beschreibung
Verweis Lastenheft	FA-11 (Kann)
User Story	Als Entwicklerin (Persona Marie) möchte ich Kommentare zu Bugs hinzufügen für Team-Kommunikation.
AC1	Eingabefeld + Submit-Button in der Detailansicht
AC2	Kommentar wird mit Autor und Zeitstempel gespeichert
AC3	Chronologische Anzeige: älteste Kommentare oben
AC4	Plain-Text: HTML-Tags werden als Text dargestellt, kein Rendering
AC5	Submit-Button deaktiviert wenn Kommentarfeld leer ist
Backend-Endpunkte	GET /api/bugs/{id}/comments POST /api/bugs/{id}/comments

5.12 FA-12 – Benutzeranmeldung (US-12)

Attribut	Beschreibung
Verweis Lastenheft	FA-12 (Muss)
User Story	Als registrierter Nutzer möchte ich mich sicher anmelden und abmelden können.
AC1	Erfolgreicher Login: Weiterleitung zur Bugliste, Session-Cookie gesetzt (HttpOnly)

AC2	Falsche Credentials: Fehlermeldung ohne Preisgabe ob Username existiert
AC3	Logout: Session invalidiert, Weiterleitung zur Login-Seite
AC4	Geschützte Seiten ohne Login: Redirect zur Login-Seite
AC5	Session-Persistenz: User bleibt nach Page-Reload eingeloggt
AC6 (Registrierung)	Nicht-eingeloggte Nutzer können einen Account selbst anlegen. Default-Rolle = TESTER. Passwort $\geq$ 8 Zeichen, BCrypt-gehasht.
Backend-Endpunkte	POST /api/auth/login POST /api/auth/logout POST /api/auth/register GET /api/auth/me

5.12 FA-12 – Benutzeranmeldung (US-12)

Session-Architektur:

Die Session-Verwaltung erfolgt nicht über Spring's eingebaute HttpSession, sondern über eine dedizierte SessionStore-Komponente. Diese hält Sessions als in-memory ConcurrentHashMap mit über SecureRandom generierten 32-Byte-Tokens (Hex-encoded). Der SessionAuthFilter (OncePerRequestFilter) liest das session-Cookie bei jedem Request, lädt die zugehörige Session aus dem Store und setzt eine Authentication in den Spring SecurityContextHolder.

Begründung: volle Kontrolle über Token-Format und Cookie-Flags, keine Abhängigkeit von spring-session-jdbc, einfacher zu testen, Token-Generation kryptographisch sicher (SecureRandom).

Bekannte Limitierungen (für Produktivbetrieb):

- Sessions liegen in-memory → bei Backend-Restart sind alle Nutzer ausgeloggt.
- Kein Multi-Instance-Support → bei horizontaler Skalierung müsste der Store auf Redis o.ä. ausgelagert werden.
- Kein Session-Timeout (TTL) implementiert; Sessions enden nur bei explizitem Logout oder Backend-Restart.

Im Rahmen des Uni-Projekts (Single-Instance-Deployment, kein 24/7-Betrieb laut 3.3) sind diese Limitierungen akzeptabel.

5.13 FA-13 – E-Mail-Benachrichtigung

Attribut	Beschreibung
Verweis Lastenheft	FA-13 (Wird nicht)
Status	Bewusst nicht im Lieferumfang, siehe Abschnitt 2.4
Begründung	Versand erfordert SMTP-Server und kollidiert mit dem Scope einer schlanken, lokal gehosteten Eigenlösung. Eine spätere Erweiterung ist über das Activity-Log technisch möglich.

5.14 FA-14 – Bug-Historie (US-14)

Attribut	Beschreibung
Verweis Lastenheft	FA-14 (Soll)
User Story	Als Projektleiterin (Persona Sandra) möchte ich alle Änderungen nachvollziehen können (wer, wann, was).
AC1	Bereich "Verlauf" in der Bug-Detailansicht sichtbar
AC2	Eintrag enthält: Zeitstempel, User, Aktion, Feldname, Alt-Wert → Neu-Wert
AC3	Erfasste Felder: Status, Priorität, Bearbeiter, Titel, Beschreibung
AC4	Sortierung: Neueste Einträge oben
AC5	Leerer Zustand: Mindestens "Bug erstellt von [Reporter]" vorhanden
Backend-Endpunkt	GET /api/bugs/{id}/activities

5.15 FA-15 – Rollen-basierte Zugriffskontrolle (US-15)

Attribut	Beschreibung
Verweis Lastenheft	FA-15 (Muss)
User Story	Als Projektleiterin (Persona Sandra) möchte ich die Rollen klar vergeben, damit Entwickler und Tester ihren zugewiesenen Funktionsumfang haben.
ADMIN	Voller Zugriff: Bug-CRUD, Statuswechsel, Bearbeiterzuweisung, Tag-Verwaltung, Rollenverwaltung
DEVELOPER	Bug-CRUD, Statuswechsel, Bearbeiterzuweisung – KEIN Tag-Management

TESTER	Bug anlegen, Bugliste anzeigen, Detailansicht – KEIN Tag-Management
AC1	Übersicht aller registrierten User mit deren aktuellen Rollen
AC2	ADMIN kann Rolle eines Users ändern
Umsetzung	Spring Security mit @PreAuthorize-Annotierungen; Rollen in User-Tabelle gespeichert

5.16 FA-16 – Tags verwalten (US-16)

Attribut	Beschreibung
Verweis Lastenheft	FA-16 (Soll)
User Story	Als Projektleiterin (Persona Sandra) möchte ich die auswählbaren Tags für einen Bug verwalten können.
AC1	Auflistung aller verfügbaren Tags (nur für ADMIN sichtbar)
AC2	Neuen Tag anlegen: Name + Farbe; Fehlermeldung wenn Name leer
AC3	Tag-Namen bearbeitbar
Zuordnung	Alle Nutzer können beim Bug-Erstellen/-Bearbeiten einen oder mehrere Tags auswählen (Multi-Select). Tags werden über die Junction-Tabelle bug_tags zugeordnet (siehe 6.6).

6. Produktdaten

Die persistenten Daten werden in einer relationalen PostgreSQL-Datenbank gespeichert. Nachfolgend ist das konzeptuelle Datenmodell beschrieben.

6.1 User-Tabelle

/D10/ (/LF FA-12, FA-15/) Benutzerdaten (erwartete Größenordnung: max. 100 Nutzer):

Feld	Typ	Beschreibung
id	BIGSERIAL PK	Eindeutige ID (auto-increment)
username	VARCHAR(50) UNIQUE	Eindeutiger Benutzername, NOT NULL
email	VARCHAR(255) UNIQUE	Eindeutige E-Mail-Adresse, NOT NULL
password_hash	VARCHAR(255)	BCrypt-Hash des Passworts, NOT NULL
role	ENUM	Werte: TESTER, DEVELOPER, ADMIN; Default: TESTER
created_at	TIMESTAMPTZ	Erstellungszeitpunkt, NOT NULL
active	BOOLEAN, DEFAULT TRUE	Soft-Delete für User

6.2 Bug-Tabelle

/D20/ (/LF FA-01 bis FA-09, FA-14/) Bug-Kerndaten (erwartete Größenordnung: max. 10.000 Bugs):

Feld	Typ	Beschreibung
id	BIGSERIAL PK	Eindeutige Bug-ID (auto-increment)
title	VARCHAR(255)	Pflichtfeld, NOT NULL
description	TEXT	Pflichtfeld, NOT NULL
status	ENUM	NEU IN_BEARBEITUNG IM_REVIEW ERLEDIGT ABGELEHNT ARCHIVIERT; Default: NEU
priority	ENUM	NIEDRIG MITTEL HOCH KRITISCH; Default: MITTEL
reporter_id	BIGINT FK → User	Automatisch gesetzter eingeloggter User
assignee_id	BIGINT FK → User	Nullable; zugewiesener Bearbeiter
archived	BOOLEAN	Default: false; Soft-Delete-Flag

created_at	TIMESTAMPZ	Erstellungszeitpunkt, NOT NULL
updated_at	TIMESTAMPZ	Letzter Änderungszeitpunkt, NOT NULL

Hinweis: Die Tag-Zuordnung erfolgt nicht direkt in der Bug-Tabelle, sondern über eine separate Junction-Tabelle bug_tags (siehe 6.6 — Bug-Tag-Junction). Damit ist eine Many-to-Many-Beziehung zwischen Bugs und Tags möglich.

6.3 Kommentar-Tabelle

/D30/ (/LF FA-11/) Kommentare zu Bugs (erwartete Größenordnung: max. 50.000 Kommentare):

Feld	Typ	Beschreibung
id	BIGSERIAL PK	Eindeutige Kommentar-ID
bug_id	BIGINT FK → Bug	Betroffener Bug, NOT NULL
user_id	BIGINT FK → User	Autor des Kommentars, NOT NULL
text	TEXT	Kommentarinhalt als Plain-Text, NOT NULL
created_at	TIMESTAMPZ	Erstellungszeitpunkt, NOT NULL

6.4 Historien-Tabelle

/D40/ (/LF FA-14/) Änderungshistorie je Bug (erwartete Größenordnung: max. 100.000 Einträge):

Feld	Typ	Beschreibung
id	BIGSERIAL PK	Eindeutige ID
bug_id	BIGINT FK → Bug	Betroffener Bug, NOT NULL
user_id	BIGINT FK → User	Auslösender User, NOT NULL
action	VARCHAR(100)	z.B. STATUS_CHANGE, FIELD_UPDATE, CREATED
field	VARCHAR(100)	Betroffenes Feld (z.B. status, priority)
old_value	TEXT	Wert vor der Änderung (nullable)
new_value	TEXT	Wert nach der Änderung
created_at	TIMESTAMPZ	Zeitstempel der Änderung, NOT NULL

6.5 Tag-Tabelle

/D50/ (/LF FA-16/) Tags und Bug-Tag-Zuordnung:

Feld	Typ	Beschreibung
id	BIGSERIAL PK	Eindeutige Tag-ID
name	VARCHAR(100) UNIQUE	Tag-Name, NOT NULL
color	VARCHAR(7)	Hex-Farbcode, z.B. #FF5733
created_at	TIMESTAMPTZ	Erstellungszeitpunkt, NOT NULL

6.6 Bug-Tag-Junction-Tabelle

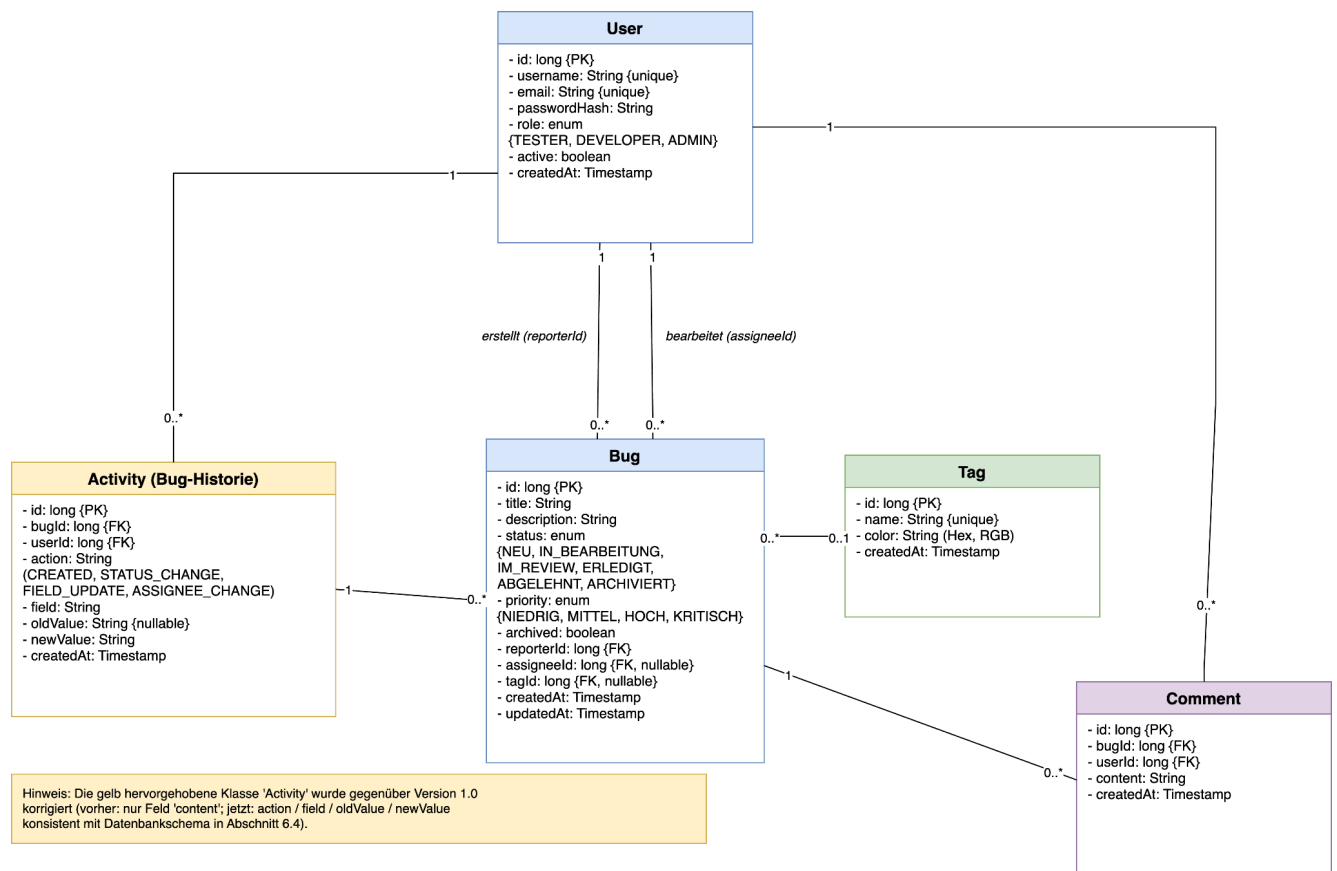
/D60/ (/LF FA-16/) Verknüpfungstabelle zwischen Bugs und Tags (Many-to-Many; erwartete Größenordnung: max. 30.000 Einträge):

Feld	Typ	Beschreibung
bug_id	BIGINT FK → Bug	Bug-ID, NOT NULL, ON DELETE CASCADE
tag_id	BIGINT FK → Tag	Tag-ID, NOT NULL, ON DELETE RESTRICT
(Composite-PK)	PRIMARY KEY (bug_id, tag_id)	verhindert duplikate Tag-Zuweisungen

Begründung der ON-DELETE-Strategien:

- bug_id ON DELETE CASCADE: bei Hard-Delete eines Bugs werden Junction-Einträge automatisch entfernt (Sicherheitsnetz).
- tag_id ON DELETE RESTRICT: ein Tag kann nicht gelöscht werden, solange er noch zugewiesen ist. Verhindert versehentlichen Datenverlust bei Tag-Verwaltung durch Admin.

6.7 Klassendiagramm



7. Produktleistungen

Leistungsanforderungen an einzelne Funktionen und Daten, mit messbaren Kriterien:

ID	Leistungsmerkmal	Messbare Anforderung	Priorität
NFA-04a	Antwortzeit Bugliste laden	< 2 Sekunden bei bis zu 100 Bugs; gemessen mit JMeter auf Standard-Laptop (16 GB RAM, Intel i5)	Soll
NFA-04b	Antwortzeit Bug anlegen/bearbeiten	< 2 Sekunden Roundtrip-Zeit; gemessen wie NFA-04a	Soll
NFA-04c	Antwortzeit Filterung/Suche	Clientseitige Filterung zeigt Ergebnisse innerhalb 300 ms nach Eingabe (Debounce)	Soll
NFA-01a	Erlernbarkeit	Neuer Nutzer kann Bug anlegen innerhalb 3 Minuten ohne externe Hilfe; Nachweis: Usability-Test mit 3 Probanden	Soll
NFA-06a	Datensicherheit	Kein Datenverlust bei normalem Betrieb; alle Schreiboperationen transaktional (Spring @Transactional)	Muss
NFA-06b	Fehlerstabilität	Unkontrollierte Abstürze dürfen nicht auftreten; alle Exceptions werden geloggt und dem Nutzer als verständliche Fehlermeldung zurückgegeben	Muss
NFA-05a	Code-Qualität	Unit-Test-Coverage in Service-Schicht $\geq 60\%$ (JaCoCo); keine Critical-Findings in SonarLint-Analyse	Soll
NFA-05b	Dokumentation	JavaDoc auf allen öffentlichen Methoden in Service- und Controller-Schicht	Soll

8. Qualitätsanforderungen

Die folgende Qualitätsmatrix orientiert sich an ISO/IEC 25010. Die Qualitätsstufen sind: sehr gut (✓✓), gut (✓), normal (○), nicht relevant (–).

Qualitätsmerkmal	sehr gut	gut	normal	nicht rel.	Priorität	NFA-Ref.
Funktionalität						
Angemessenheit		✓			Muss	NFA-02
Richtigkeit	✓✓				Muss	NFA-06
Interoperabilität			○		Soll	NFA-08
Sicherheit	✓✓				Muss	NFA-07
Zuverlässigkeit						
Reife		✓			Muss	NFA-06
Fehlertoleranz			○		Muss	NFA-06
Wiederherstellbarkeit			○		Muss	NFA-06
Benutzbarkeit						
Verständlichkeit		✓			Soll	NFA-01
Erlernbarkeit	✓✓				Soll	NFA-01
Bedienbarkeit		✓			Soll	NFA-01
Effizienz						
Zeitverhalten		✓			Soll	NFA-04
Ressourcenverhalten			○		Kann	–
Wartbarkeit						
Analysierbarkeit		✓			Soll	NFA-05
Änderbarkeit		✓			Soll	NFA-05
Testbarkeit		✓			Soll	NFA-05

9. Benutzungsoberfläche

9.1 Allgemeine Gestaltungsrichtlinien

- /B10/ Die Oberfläche folgt dem Material-Design-ähnlichen Gestaltungsregelwerk von React.
- /B20/ Navigation: Horizontale Navigationsleiste mit Logo, Hauptmenüpunkten und Nutzer-Dropdown (Logout).
- /B30/ Alle interaktiven Elemente sind per Maus bedienbar; Tastaturnavigation für Barrierefreiheit wünschenswert.
- /B40/ Fehlermeldungen werden inline am verursachenden Feld sowie als Toast-Notification oben rechts angezeigt.
- /B50/ Die Oberfläche ist responsiv für Desktop-Bildschirme (≥ 1280 px Breite) ausgelegt; mobile Optimierung ist nicht vorgesehen.
- /B60/ Ladezustände werden durch Skeleton-Loader oder Spinner signalisiert.

9.2 Hauptmasken

- Login-Seite: Formular mit Feldern Benutzername und Passwort, Anmelde-Button, generische Fehlermeldung bei Misserfolg.
- Bugliste: Tabellarische Übersicht mit Spalten ID, Titel, Status (farbig), Priorität (farbig), Bearbeiter, Erstellt-am. Filterleiste (Status) und Suchfeld oben. Button "Neuer Bug" prominent platziert.
- Bug-Detailansicht: Strukturierte Ansicht aller Bug-Felder. Abschnitte: Grunddaten, Status & Priorität, Bearbeiter, Tags, Kommentare, Änderungshistorie. Aktionsbuttons: Bearbeiten, Archivieren (rollenabhängig sichtbar).
- Bug-Formular (Anlegen/Bearbeiten): Felder: Titel (Text, Pflicht), Beschreibung (Textarea, Pflicht), Priorität (Dropdown), Status (Dropdown, Übergänge validiert), Bearbeiter (Dropdown), Tags (Multi-Select). Speichern- und Abbrechen-Button.
- Tag-Verwaltung (nur ADMIN): Liste aller Tags mit Name und Farbvorschau. CRUD-Aktionen je Tag.

9.3 Rollenbasierte Sichtbarkeit

Rolle	Zugängliche Funktionen	Beschreibung
TESTER	FA-01, FA-02, FA-03, FA-09, FA-10, FA-11	Bugs anlegen, anzeigen, kommentieren, suchen/filtern
DEVELOPER	FA-01 bis FA-11, FA-14 (außer FA-16)	Alles wie TESTER + Bugs bearbeiten, archivieren, Bearbeiter zuweisen
ADMIN	FA-01 bis FA-16 (alle)	Vollzugriff inkl. Benutzerverwaltung, Tag-Verwaltung, Bug ablehnen

10. Nichtfunktionale Anforderungen

Alle Anforderungen, die sich nicht auf Funktionalität, Leistung oder Benutzungsoberfläche beziehen:

ID	Kategorie	Anforderung	Priorität
NFA-01	Benutzbarkeit	Ein neuer Nutzer mit grundlegender PC-Erfahrung muss innerhalb von 3 Minuten ohne externe Hilfe einen Bug anlegen können. Nachweis: Usability-Test mit 3 Probanden.	Soll
NFA-02	Technologie	Backend: Java 21, Spring Boot 3.x (Spring Web, Spring JDBC mit NamedParameterJdbcTemplate, Spring Security). Frontend: React 18+, TypeScript, Vite. Build: Maven (Backend), npm/Vite (Frontend). DB-Migrationen: Flyway. Keine Abweichung ohne Genehmigung.	Muss
NFA-03	Plattform	Die Anwendung läuft auf einem Standard-PC und wird ausschließlich über einen modernen Webbrowser (Chrome 120+, Firefox 120+, Edge 120+) bedient.	Muss
NFA-04	Antwortzeit	Einfache Aktionen (Anlegen, Anzeigen, Bearbeiten) antworten in < 2 Sekunden bei bis zu 100 Bugs. Messung mit JMeter auf Standard-Laptop (16 GB RAM, Intel i5+).	Soll
NFA-05	Wartbarkeit	Unit-Test-Coverage Service-Schicht $\geq 60\%$ (JaCoCo). SonarLint: keine Critical-Findings. JavaDoc auf allen öffentlichen Methoden in Service- und Controller-Schicht.	Soll
NFA-06	Zuverlässigkeit	Die Anwendung darf bei normaler Nutzung nicht unkontrolliert abstürzen. Alle Schreiboperationen sind transaktional. Daten dürfen nicht verloren gehen.	Muss
NFA-07	Sicherheit	Passwörter: BCrypt-Hash (Cost-Faktor 10). Session-Cookies: HttpOnly + SameSite=Lax (Secure-Flag im Production-Deployment via HTTPS). SQL-Injection: ausgeschlossen durch ausschließliche Nutzung von PreparedStatements (Spring JDBC mit NamedParameterJdbcTemplate). Schutz vor CSRF durch HttpOnly-Cookies + SameSite=Lax + CORS-Whitelist (kein Token-basierter CSRF-Schutz, da klassische CSRF-Token-Mechanismen für REST-API mit SPA-Frontend redundant sind).	Muss
NFA-08	Architektur	Strikte Trennung von Backend (REST-API) und Frontend (SPA). Kommunikation ausschließlich über JSON-REST. Frontend ist unabhängig deploybar. Kein Server-Side-Rendering.	Muss

11. Technische Produktumgebung

11.1 Software

Server-Betriebssystem: Linux (Ubuntu 22.04 LTS oder kompatibel) oder Windows 10/11 für lokalen Betrieb.

Client-Betriebssystem: Beliebig (Windows 10/11, macOS 13+, Ubuntu 22.04+), Zugriff via Browser.

Komponente	Technologie / Version	Zweck
Backend-Runtime	Java 21 (OpenJDK)	Laufzeitumgebung
Backend-Framework	Spring Boot 3.x	REST-API, Security, JDBC
Frontend-Framework	React 18+ mit TypeScript	SPA / UI
Frontend-Build	Vite	Build-Tool für Frontend
Datenbank	PostgreSQL 15+	Persistente Datenspeicherung
Build-Tool Backend	Maven 3.9+	Build, Dependency-Management
Versionsverwaltung	Git	Source-Code-Management
CI/CD	Git Actions	Automatisierte Tests, Build
Test-Framework	JUnit 5, Spring Boot Test	Unit- und Integrationstests
Performance-Test	JMeter	Antwortzeitmessung (NFA-04)
Code-Qualität	SonarLint, JaCoCo	Qualitätsanalyse (NFA-05)

11.2 Hardware

- Server: Standard-PC oder Laptop (min. 8 GB RAM, Intel i5 oder vergleichbar, 20 GB freier Speicher).
- Client: Gerät mit Webbrowser und Grafikbildschirm (min. 1280×800 px Auflösung).
- Netzwerk: Lokales Netzwerk oder localhost; keine Internetverbindung für den Betrieb erforderlich.

11.3 Orgware

- MCI-Git-Repository: Private Repo der Projektgruppe.
- Git Actions: Für automatisierte Build- und Test-Pipelines.
- Keine externe Cloud-Infrastruktur; alle Komponenten laufen lokal oder im Hochschulnetz.

11.4 Produktschnittstellen

- Frontend → Backend: REST-API über HTTP/HTTPS, JSON-Format, Port 8080 (Backend), Port 5173 (Frontend Dev) bzw. statische Dateien (Prod).
- Backend → Datenbank: JDBC-Verbindung zu PostgreSQL, Port 5432.
- Keine externen Schnittstellen zu Drittsystemen (bewusste Abgrenzung, vgl. Abschnitt 2.4).

12. Spezielle Anforderungen an die Entwicklungsumgebung

Die Entwicklungsumgebung entspricht weitgehend der Produktumgebung:

- IDE: IntelliJ IDEA (empfohlen) oder VS Code mit Java-Extension Pack.
- Datenbankentwicklung: PostgreSQL lokal oder via Docker; alternativ H2 in-memory für schnelle Tests.
- Frontend-Entwicklung: Node.js 20 LTS, npm, Vite Dev-Server (Port 5173) mit HMR.
- API-Testing: Postman oder IntelliJ HTTP-Client für manuelle REST-Tests.
- Quellcode: Git mit Feature-Branch-Workflow; Pull Requests auf develop-Branch. Branch-Namenskonvention: feature/T0XX-kurzbeschreibung. Merge-Strategie: Squash-Merge zu develop (1 Commit pro Task). Releases erfolgen über Merge develop → main mit Tag (v0.5, v1.0). Mindestens ein Review pro PR durch ein anderes Teammitglied.
- Keine Abweichungen von der Produktumgebung in Bezug auf Laufzeit (Java 21) und Framework-Versionen (Spring Boot 3.x, React 18+).

13. Gliederung in Teilprodukte (Release-Planung)

Release	Zieldatum	Enthaltene FA	Beschreibung
MVP v0.1	bis 09.05.2026	FA-01, FA-02, FA-03, FA-12	Kern-CRUD: Bugs anlegen, anzeigen, anmelden/abmelden
Beta v0.5	bis 16.05.2026	+ FA-04, FA-05, FA-06, FA-15	Bug bearbeiten, archivieren, Status-Machine, Rollen
Release v1.0	bis 22.05.2026	+ FA-07, FA-08, FA-09, FA-14, FA-16	Vollumfang: Priorität, Bearbeiter, Filter, Historie, Tags
Optional v1.1	nach 22.05.2026	FA-10, FA-11	Freitextsuche, Kommentare (wenn Zeit verbleibt)

14. Abnahmekriterien

14.1 Funktionale Abnahme

- Alle Muss-Anforderungen (FA-01, FA-02, FA-03, FA-04, FA-05, FA-06, FA-12, FA-15) implementiert und in Live-Demo fehlerfrei vorführbar
- Anwendungsfälle UC-01 bis UC-06 laufen in der Live-Demo ohne Fehler durch
- Alle Soll-Anforderungen zu mindestens 80 % implementiert

14.2 Qualitative Abnahme

- Usability-Tests von allen Teammitgliedern ohne kritische Auffälligkeiten bestanden

14.3 Dokumentation

- Quellcode vollständig im MCI-Git-Repository eingecheckt
- README.md mit Setup-Anleitung und Architekturskizze vorhanden
- Lastenheft v1.0 und Pflichtenheft v1.1 als PDF im Sakai abgegeben

14.4 Präsentation & Bericht

- Abschlusspräsentation am 22.05.2026 erfolgreich durchgeführt
- Projektbericht (mind. 10 Seiten) eingereicht bis 22.05.2026, 23:59 Uhr als PDF

15. Ergänzungen

15.1 Annahmen und Abhängigkeiten

- Es wird davon ausgegangen, dass alle Teammitglieder Grundkenntnisse in Java, Spring Boot und React mitbringen.
- Die Datenbankmigrationen werden mit Flyway verwaltet (Entscheidung beim Implementierungsstart getroffen, siehe T010). Migrations-Dateien liegen in backend/src/main/resources/db/migration/ mit dem Namensschema V<N>__<beschreibung>.sql.
- Für die Demonstration und lokale Bereitstellung wird das Frontend entweder über den Vite-Preview-Server oder einen lokal installierten statischen Webserver (z. B. Nginx) ausgeliefert. Ein produktives Hosting im Internet ist nicht Bestandteil des Projekts (vgl. Lastenheft Abschnitt 5.2).
- Für die Demo sind mindestens 5 Beispiel-Bugs mit unterschiedlichen Status und Prioritäten in der Datenbank vorhanden.

15.2 Offene Punkte

- Die genaue Aufteilung der ADMIN-Funktionen (Benutzerverwaltung) auf dem Frontend ist noch zu konkretisieren.
- Das genaue Farbschema der Prioritäten (Hex-Codes) wird im UI-Design-Sprint festgelegt.

15.3 Lieferumfang

Liefergegenstand	Format / Ablageort
Lauffähige Webanwendung BugTracker	Spring Boot JAR + React-Build (statische Dateien)
Quellcode	MCI-Git-Repository (Gruppe DE 3)
Lastenheft v1.1	Word- und PDF-Dokument
Pflichtenheft v1.1 (dieses Dokument)	Word- und PDF-Dokument
MoSCoW-Matrix	Excel-Datei (.xlsx)
README.md mit Setup-Anleitung	Markdown im MCI-Git-Repository
Unit- und Integrationstests	JUnit 5 / Spring Boot Test im Quellcode
CI/CD-Pipeline-Konfiguration	Git Actions YAML im Repository
Abschlusspräsentation	PowerPoint / PDF, Termin: 22.05.2026
Projektbericht	Word- oder PDF-Dokument, Abgabe: 22.05.2026, 23:59 Uhr